

ZAP-RNS: Cryptographic Name–Identity Binding for In-Cluster Service Discovery

ZAP Protocol Authors
zap-proto.io

2026

Abstract

We present ZAP-RNS, a service-naming scheme in which every name is bound at registration time to a long-term key encapsulation mechanism (KEM) public key. Claims about a name — its existence, its current public key, its metadata — are signed under the corresponding private key, and lookups return a verifiable chain rooted at a per-cluster authority public key. This eliminates name spoofing and Sybil attacks at the protocol layer: spoofing the name N reduces to recovering the KEM private key for N or forging a signature under it. We compare ZAP-RNS against DNS, DNSSEC, mDNS, and SPIFFE/SPIRE; give a formal model in which the name-spoofing advantage is bounded by KEM key-recovery and signature forgery probabilities; and discuss the operational properties of caching, revocation, and federation. ZAP-RNS is post-quantum native: its binding primitive is a KEM (ML-KEM-768 by default) and its signature primitive is composable with ML-DSA-65, so the scheme inherits PQ guarantees without retrofit. We argue ZAP-RNS is the right primitive for in-cluster service mesh discovery because identity is checked at every lookup rather than only at TLS handshake, and because the trust root is small, audit-able, and pluggable into existing workload-attestation infrastructure.

1 Introduction

Service discovery has accumulated layers of compromise. The Domain Name System [7] answers “where is N ?” but not “is the holder of N who I think it is?” DNSSEC [1] signs records but deploys end-to-end in well under 30% of the public Internet and almost never inside a cluster. Multicast DNS [2] and SRV records [5] push naming further into the local network with no identity binding at all: any host on the segment can claim any name.

Cluster-scale systems — ZooKeeper, etcd, Consul, Kubernetes — collapse name and address into a key/value store guarded by an out-of-band authentication tier. Discovery is done by querying the store; the identity of the discovered service is not part of the record but re-established later via TLS, mTLS, or workload attestation. This is the SPIFFE/SPIRE model [11]: a name (URI SVID) is issued as an X.509 certificate by a workload attestor on the assumption that the Kubernetes API server, kubelet, and node identity are trusted. The identity binding lives in a separate plane from the discovery plane.

The recurring failure across these designs is the same: *the name itself does not commit to a key*. To go from name to authenticated endpoint, every system relies on at least one out-of-band step — DNSSEC chain, X.509 chain, mesh CA, SPIFFE attestor — and the operational burden of those steps is what causes them to be skipped in practice.

We propose ZAP-RNS, a scheme in which the name *is* a commitment to a public key. Registration produces a record $(N, pk_N^{\text{KEM}}, pk_N^{\text{Sig}}, ttl, \sigma)$ where σ is a chain of signatures terminating at a single per-cluster RNS-root public key. Lookups return the full record. Connections encrypt to pk_N^{KEM} , so reaching an impostor requires either knowing sk_N^{KEM} or compromising the RNS

root. The same record is verifiable offline by any client that has been provisioned with the root once. Identity, naming, and key distribution become a single primitive.

The contributions of this paper are:

- (1) A formal definition of ZAP-RNS records, registration, and lookup (§2).
- (2) A security model and reductions showing that name-spoofing advantage is bounded by KEM key-recovery, signature forgery, and RNS-root compromise (§3, §4).
- (3) A Sybil-resistance argument tying registration cost to the cluster’s chosen workload-attestation primitive (§5).
- (4) A comparison of ZAP-RNS with DNS, DNSSEC, mDNS, and SPIFFE on six axes including post-quantum readiness (§6).
- (5) Implementation notes on caching, revocation, and multi-authority federation (§7).

2 Definition of ZAP-RNS

Let $\text{KEM} = (\text{KEM.KGen}, \text{KEM.Encap}, \text{KEM.Decap})$ be an IND-CCA2 secure key encapsulation mechanism and let $\text{Sig} = (\text{Sig.KGen}, \text{Sig.Sign}, \text{Sig.Verify})$ be an EUF-CMA secure signature scheme. Default choices are ML-KEM-768 [9] and ML-DSA-65 [10], but the scheme is parametric in the primitives.

Definition 1 (Record). A ZAP-RNS record is a tuple

$$\text{rec} = (N, pk^{\text{KEM}}, pk^{\text{Sig}}, t_0, ttl, m, \sigma_{\text{owner}}, \sigma_{\text{auth}})$$

where $N \in \{0,1\}^*$ is the human-readable name, pk^{KEM} and pk^{Sig} are the long-term KEM and signing public keys, t_0 is issuance time, ttl is validity duration, m is auxiliary metadata (endpoint, attestation evidence), $\sigma_{\text{owner}} = \text{Sig.Sign}(sk^{\text{Sig}}, H(N \| pk^{\text{KEM}} \| pk^{\text{Sig}} \| t_0 \| ttl \| m))$ is the self-signed binding, and $\sigma_{\text{auth}} = \text{Sig.Sign}(sk_{\text{root}}, H(\text{rec} \setminus \sigma_{\text{auth}}))$ is the authority counter-signature.

Trust root. A cluster operator generates a single $(pk_{\text{root}}, sk_{\text{root}})$ pair. pk_{root} is bundled into every workload’s image or sidecar at provisioning time. sk_{root} lives in an HSM or threshold service. There is exactly one root per trust domain, and it is the single point of human-auditable trust.

Registration. A workload runs the protocol of Algorithm 1. It generates KEM and signing keypairs, constructs an unsigned record, signs it under sk^{Sig} , and submits the record together with attestation evidence (e.g., a Kubernetes ServiceAccount token, a TPM quote, a TEE attestation) to the RNS authority. The authority validates evidence under its policy, checks that N is not already registered (or that the requester is authorised to roll the key), and emits σ_{auth} .

Lookup. A client queries the RNS authority for N and receives rec . Verification:

1. Check $t_0 \leq \text{now} \leq t_0 + ttl$.
2. Check $\text{Sig.Verify}(pk^{\text{Sig}}, H(r), \sigma_{\text{owner}})$.
3. Check $\text{Sig.Verify}(pk_{\text{root}}, H(r \| \sigma_{\text{owner}}), \sigma_{\text{auth}})$.
4. Consult the freshness oracle (revocation list or short-lived witness) for rec .

On success the client treats pk^{KEM} as the only legitimate encapsulation target for N .

Algorithm 1 Registration

- 1: $(pk^{\text{KEM}}, sk^{\text{KEM}}) \leftarrow \text{KEM.KGen}()$
 - 2: $(pk^{\text{Sig}}, sk^{\text{Sig}}) \leftarrow \text{Sig.KGen}()$
 - 3: $r \leftarrow (N, pk^{\text{KEM}}, pk^{\text{Sig}}, t_0, \text{ttl}, m)$
 - 4: $\sigma_{\text{owner}} \leftarrow \text{Sig.Sign}(sk^{\text{Sig}}, H(r))$
 - 5: submit $(r, \sigma_{\text{owner}}, \text{evidence})$ to RNS authority
 - 6: authority verifies evidence and $\text{Sig.Verify}(pk^{\text{Sig}}, H(r), \sigma_{\text{owner}})$
 - 7: authority emits σ_{auth} and stores $\text{rec} = (r, \sigma_{\text{owner}}, \sigma_{\text{auth}})$
-

Connection. The client encapsulates a fresh shared secret under pk^{KEM} and sends the ciphertext together with its first record. Only the holder of sk^{KEM} for N can decapsulate. Subsequent application traffic uses the derived secret in an AEAD. Identity is checked *on every lookup*, not only at handshake.

Encrypted lookup. The lookup itself can be encapsulated to pk_{root} (or a sub-key derived from it via HPKE), so that passive observers see only an encrypted blob between client and RNS. This degrades worst-case adversary capability from “learns who talks to whom” to “learns who talks to RNS”.

3 Security Properties

Authenticity. A name not registered in RNS cannot be served. A client receiving a record for N verifies a signature chain to pk_{root} ; absent the root key, no adversary can fabricate a record that verifies.

Continuity. Once N is bound to pk_N^{KEM} , the binding changes only via a fresh authority counter-signature. The authority policy decides whether re-binding requires (a) a signature from the old sk^{Sig} (Lamport-style continuity [6]), (b) re-running the original attestation, or (c) explicit operator approval. ZAP-RNS encodes the policy choice in m and refuses lookups whose continuity chain breaks the policy.

Privacy. With encrypted lookup, a network observer learns neither the names a client resolves nor the public keys returned. The authority still observes both, mitigated by classic PIR [3] or OHTTP-style [12] interposition. We treat scalable privacy-preserving lookup as an open problem (§8).

Freshness and revocation. TTLs are short (default 300s). Each authority publishes a signed Merkle witness of currently-valid records every $\text{TTL}/2$; clients consult it at each lookup. Compromised KEM keys are revoked by appending a tombstone record signed by the authority.

4 Formal Model

We define the name-spoofing experiment $\text{Exp}_{\text{ZAP-RNS}}^{\text{spooF}}(\mathcal{A})$:

1. Challenger generates $(pk_{\text{root}}, sk_{\text{root}})$ and gives pk_{root} to $\mathcal{A}$.
2. $\mathcal{A}$ may query a registration oracle on names of its choice; the oracle runs Algorithm 1 honestly and returns the resulting records and the public keys, but *not* the private keys.

3. $\mathcal{A}$ outputs a target name N^* (not previously registered by $\mathcal{A}$ as the legitimate owner) and a forged record rec^* .
4. $\mathcal{A}$ wins if rec^* verifies against pk_{root} and rec^* 's public key differs from the public key on file at the challenger for N^* (or no record exists for N^*).

Theorems 1–3 of the companion proof file bound

$$\text{Adv}_{\text{ZAP-RNS}}^{\text{spoof}}(\mathcal{A}) \leq \text{Adv}_{\text{Sig}}^{\text{EUF-CMA}}(\mathcal{B}_1) + \text{Adv}_{\text{KEM}}^{\text{key-rec}}(\mathcal{B}_2) + \Pr[\text{root compromise}],$$

where the third term is policy-controlled and audit-able.

Connection binding. Theorem 2 of the proof file shows that under IND-CCA2 of KEM, a client encrypting to name N^* reaches a peer holding $sk_{N^*}^{\text{KEM}}$ with overwhelming probability, or learns nothing.

5 Sybil Resistance

ZAP-RNS reduces Sybil resistance to attestation-bypass cost. The authority emits σ_{auth} only after attestation evidence satisfies its policy. The policy is pluggable:

- **K8s ServiceAccount.** The cluster's API server is the attester; cost = stealing a ServiceAccount token.
- **TPM quote.** A measured-boot TPM signs a PCR digest; cost = compromising a hardware root of trust.
- **TEE attestation.** An SEV-SNP / TDX / SGX quote; cost = breaking the TEE.
- **Proof-of-stake.** Each registration burns or stakes a token; cost = direct economic.

The Sybil bound is tight: an adversary capable of n legitimate registrations under the policy can bind n names. The cluster operator chooses the per-registration cost to match the consequences. There is no ambient Sybil channel because the only way to insert a record is through the attester.

6 Comparison

Table 1 compares discovery schemes on six properties: intrinsic name-to-key binding, lookup-time authentication of records, encryption-to-name (a client can encrypt directly to a returned identity), post-quantum readiness, deployment effort relative to a bare DNS deployment, and resistance to a passive network observer learning the names being resolved.

ZAP-RNS is the only scheme that ticks every column. The closest analogue is the Host Identity Protocol (HIP) [8], which also separates a host identity from its locator and binds names to public keys. ZAP-RNS differs in four ways: (i) it is post-quantum native; (ii) lookups return a chain to a single auditable root, not arbitrary DNS resource records; (iii) registration is gated by pluggable attestation rather than the open Internet PKI; (iv) the binding is to a KEM key, enabling encryption-to-name without a separate key-exchange roundtrip.

7 Implementation Notes

Caching. Records are immutable for their TTL. Clients cache verified records and a freshness witness; a cache hit costs one signature verification. Negative caching of NXDOMAIN responses, signed by the authority, prevents downgrade-to-no-record attacks.

	Name binds key	Auth lookup	Enc-to-name	PQ ready	Op cost	Lo
DNS [7]	no	no	no	n/a	low	
DNSSEC [1]	no*	yes	no	retro	high	
mDNS [2]	no	no	no	n/a	low	
SRV [5]	no	no	no	n/a	low	
SPIFFE [11]	X.509	at TLS only	no	retro	high	
HIP [8]	yes (HI)	yes	yes	retro	high	
Tor onion [4]	yes (v3)	yes	yes	no	high	
ZAP-RNS (this work)	yes	yes (every lookup)	yes	native	med	yes

Table 1: Comparison of service discovery schemes. “Name binds key”: the name is a commitment to a public key. “Auth lookup”: lookups return signed records that any client can verify. “Enc-to-name”: clients can encrypt directly to the returned identity without a preceding handshake. *DNSSEC binds the *zone* to a key, not individual service names to service keys.

Revocation. Short TTLs (60s to 300s) replace explicit revocation in the common case. For emergency revocation the authority publishes a signed tombstone with the same name, which evicts cached records on next freshness check.

Federation. Multiple authorities co-exist by being placed in a forest: each authority A_i has $pk_{\text{root},i}$, and clients hold a known-roots set. Names are namespaced by authority ($N@A_i$), so collisions are impossible. Cross-authority delegation is a counter-signature on a delegation record, identical in shape to σ_{auth} .

Operational primitives. The authority is a small stateful service: ~ 10 kB per record, $O(1)$ verification per lookup, and no external dependencies beyond the HSM holding sk_{root} . For a cluster of 10^5 services the on-disk state is below 1 GB and fits in memory on a single node.

Sidecar integration. A workload’s sidecar exposes a local UDS, performs registration on startup, caches its own KEM private key in a tmpfs (or HSM), and intercepts outgoing connections to translate N into $(pk_N^{\text{KEM}}, \text{endpoint})$. No application code change is required; legacy clients see a familiar resolver interface.

8 Open Questions

Lookup privacy at scale. Encrypting the lookup to the authority hides names from network observers, but the authority itself still sees them. For large public clusters one wants a private information retrieval [3] or oblivious-HTTP [12] overlay. Both are quadratic-or-worse in the worst case. We have not benchmarked PIR-over-RNS in this paper.

Revocation latency. Short TTLs reduce revocation latency but inflate authority load. We conjecture an adaptive TTL keyed to a service’s request rate is workable, but have not analysed the adversarial-load model.

Multi-region authority placement. Federated authorities trade global consistency for partition tolerance. CRDT-based authority sets are an active area; we leave them future work.

Hybrid signatures. ML-DSA-65 is a sound default, but a hybrid (ML-DSA + Ed25519) registration signature would tolerate a catastrophic break in either scheme. The cost is roughly doubled record size; we treat this as a configuration parameter.

9 Conclusion

ZAP-RNS makes the name into a key. The reduction is sharp: spoofing N requires breaking a KEM, forging a signature, or compromising a single audit-able root. The scheme is post-quantum native, integrates with any workload-attestation primitive a cluster already runs, and adds at most one signature verification per lookup beyond a DNS-style cache. For in-cluster service mesh discovery — where the trust domain is small, attestation is already required, and identity must be checked at every hop — ZAP-RNS is a strict improvement on the status quo of layered DNS, X.509, and SPIFFE.

References

- [1] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. DNS security introduction and requirements. RFC 4033, IETF, 2005.
- [2] S. Cheshire and M. Krochmal. Multicast DNS. RFC 6762, IETF, 2013.
- [3] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan. Private information retrieval. In *FOCS*, 1995.
- [4] R. Dingledine, N. Mathewson, and P. Syverson. Tor: The second-generation onion router, 2004.
- [5] A. Gulbrandsen, P. Vixie, and L. Esibov. A DNS RR for specifying the location of services (DNS SRV). RFC 2782, IETF, 2000.
- [6] L. Lamport. Password authentication with insecure communication. *Communications of the ACM*, 24(11):770–772, 1981.
- [7] P. Mockapetris. Domain names — implementation and specification. RFC 1035, IETF, 1987.
- [8] R. Moskowitz, T. Heer, P. Jokela, and T. Henderson. Host identity protocol version 2 (HIPv2). RFC 7401, IETF, 2015.
- [9] NIST. FIPS 203: Module-Lattice-Based key-encapsulation mechanism standard, 2024.
- [10] NIST. FIPS 204: Module-Lattice-Based digital signature standard, 2024.
- [11] SPIFFE Authors. SPIFFE: Secure production identity framework for everyone. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>, 2024.
- [12] M. Thomson and C. A. Wood. Oblivious HTTP. RFC 9458, IETF, 2024.